Deep Learning Methodology for Charging Management Applications in Battery Cells Based on Neural Networks

Rolando Antonio Gilbert Zequera, *Student Member, IEEE*, Viktor Rjabtšikov, *Member, IEEE*, Anton Rassõlkin, *Senior Member, IEEE*, Toomas Vaimann, *Senior Member, IEEE*, and Ants Kallaste, *Senior Member, IEEE*

***Abstract*—A Battery Energy Storage System (BESS) plays an important role in achieving energy transition and climate change mitigation, with charging management applications being a crucial topic to improve the build, design, and operation of renewable technologies. With the continuous development of Artificial Intelligence (AI), implementing accurate algorithms that monitor Key Performance Indicators (KPIs) and provide predictive maintenance is a callenging task. This article presents a solid and robust Deep Learning methodology based on Neural Networks (NNs) in the TensorFlow framework and using Python as a programming language, all to predict the Open Circuit Voltage (OCV) and improve the state estimation of battery cells. Extensive tests on Lithium-ion cells under diverse operating conditions were carried out, with data acquisition meticulously recorded using a programmable DC Electronic Load. Various architectures were designed using Keras as a high-level Application Programming Interface (API) to build Artificial Neural Networks (ANNs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs). In addition, advanced computer science techniques were executed to improve the performance of the algorithms, such as cross-validation, Fine-tuning, Regularization, Bayesian optimization, Machine Learning techniques, and Data-Driven approaches. The resulting network architectures were stored in Hierarchical Data Format (HDF5) files, tested against both seen and unseen data to ensure its robustness and effectiveness in new battery measurements. The Deep Learning methodology provides remarkable testing accuracy of over 95% for all types of NNs, affirming its high adaptability and reliability in the development of AI-powered technology for battery management.**

***Index Terms*—Deep Learning, Neural networks, Battery energy storage system (BESS).**

I. Introduction

IN THE rapidly landscape of renewable energy integration, optimal implementation of a Battery Energy Storage Systems (BESS) is essential to reduce CO2 emissions and promote climate change mitigation. Battery modeling highlights a remarkable understanding of not only the operating mechanism but also the scientific framework, which is critical for accurate diagnostics and maintenance. Different approaches have been considered to implement an optimal battery modelling, being Electrochemical model, Equivalent Circuit Model (ECM), and Mathematical models the most important to simulate, monitor, and predict Health and Charge indicators of a BESS [1], [2]. With the continuous advances of Artificial Intelligence (AI), computer science algorithms must be developed, verified, and implemented to support the knowledge provided by energy analysts in both the industrial and academic fields. Several researchers and scholars have proposed Data Science techniques that achieve the modeling and diagnostics of a BESS, which offer beneficial applications for various topics of strategic importance such as digital twins, enterprise testing and business consulting [3], [4], however, it is necessary point out the importance of strengthening a methodology that explains both the physics of a system and the behavior of Key Performance Indicators (KPIs) through AI-powered technology.

Within the framework of a BESS operation, the State of Charge (SOC) plays a fundamental role in improving the performance of renewable technologies, so charging management is a key benefit to monitor efficiency and optimize lifetime. Experimental measurements and battery testing monitor the charging and discharging processes performed by a BESS based on chemical structure, manufacturing characteristics and actual use, which are directly influenced by user needs and the type of application in the output capacity and voltage level. Furthermore, in recent years battery models based on electrochemical and engineering approaches have been proposed to develop charge profiles for different industrial applications, highlighting the impact of KPIs on the design of intelligent algorithms, being current rate and threshold voltage vital in the final charging performance [5].

In a Data-Driven context, it is essential to highlight the critical role that KPIs play in determining both their level of correlation and the final performance on the predicted variable during model evaluation. Deep Learning encompasses complex Machine Learning algorithms and advanced computing techniques based on AI and mathematical principles. These strategies enable precise management of input variables, optimizing the predictions across a wide network of neurons. However, even if the algorithm provides a high level of testing

Manuscript received 16 May 2024; revised 31 May 2024; accepted 17 June 2024. This work was supported by Estonian Research Council under Grant PSG453. *(Corresponding author: Rolando Antonio Gilbert Zequera.)*

The authors are with the Department of Electrical Power Engineering and Mechatronics, Tallinn University of Technology, Tallinn 19086, Estonia (e-mail: rogilb@taltech.ee; viktor.rjabtsikov@taltech.ee; anton.rassolkin@taltech.ee; toomas.vaimann@taltech.ee; ants.kallaste@taltech.ee).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TIV.2024.3417216.

Digital Object Identifier 10.1109/TIV.2024.3417216

2379-8858 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

accuracy, there is a gap between computational complexity and state estimation based on network architecture and BESS parameters during the implementation of AI methods [1], [3], [4], [5].

During the implementation of Deep Learning algorithms in a BESS domain, the opportunity exists to train and fine-tune a Neural Network (NN), thus optimizing the battery's operations based on Health and Charge indicators. However, when different types of datasets are collected, the challenge lies in unraveling its interpretation to achieve both Feature Engineering and Feature Selection, as battery specialists typically withhold this information. This opacity regarding the internal mechanisms of the battery presents a significant obstacle when seeking to understand, potentially modify, and test Deep Learning algorithms to interpret the predictors and the target variable during model evaluation. As highlighted by Gilbert Zequera [6], this lack of transparency poses a substantial obstacle in the pursuit of the capability, robustness, and adaptability of AI methods in battery management, which introduces the research question of designing an optimal algorithm whose mechanism can explain the behavior of physical and virtual entities from an energy framework.

Indeed, in such a context, unifying the domains of Deep Learning and a BESS becomes highly applicable and valuable. Deep Learning enables computational models to learn features progressively from data at multiple levels [7]. Compared to traditional Machine Learning methods, it has more accuracy and has a strong learning ability to use datasets for feature extraction. It has gained global attention recently as a critical practice in several industries such as Natural Language Processing, Image Recognition and Computer Vision, Recommended Systems, Autonomous Vehicles, Speech Recognition and Voice Assistant, etc. [8].

Deep Learning find valuable applications in the BESS sector, where they help dissect and comprehend the inner workings of battery components and systems, ultimately driving advancements, innovations, and optimizations in the battery modelling, state estimation, and state prediction. Li et al. [9] explored the applications of some relevant Deep Learning algorithms for SOC and State of Health (SOH) predictions, using public datasets such as NASA, CALCE, UCL, TRI, Oxford, etc. Tian et al. [10] contributed a theoretical Deep Learning framework for SOC estimation in Lithium-ion batteries, explaining recent advances and future perspectives in different battery management tasks.

Existing models integrating both Machine Learning and Deep Learning in the BESS domain have been designed to achieve battery charge and life estimation, however, robustness of the algorithm design is fundamental in the development of a methodology for the diagnostics of a BESS based on digital representations, so the integration of new tools such as Data Engineering, IT operations, and Software Development is highly valuable for both industry and academia

Relevant academic sources published by renowned scientists have proposed the incorporation of AI methods for energy demand in charging applications, whose notable contribution has been proven through probabilistic models and Deep Learning approaches [11], [12]. In 2022, Li et al. developed a Reinforcement Learning-assisted Deep Learning approach to forecast an electric vehicle charging station (EVCS), which was complemented with a Markov decision process and a LSTM algorithm [13]. Later, in 2023, Xiong and Zhou [14] combined multiple Deep Learning models to achieve forecasting in electric vehicle charging stations, and Yang et al. designed a hierarchical forecasting model to predict plug-in electric vehicles (PEV) charging by using a new attention-based LSTM Deep Learning approach [15].

TABLE I
LEV50N BATTERY CELL PARAMETERS

Parameter	Value	Unit
Nominal voltage	3.75	V
Operating voltage range	2.75 to 4.1	V
1-hr rate typical Capacity 25°C	50	Ah
Charge voltage limit at 25°C	4.1	V
Charge termination threshold current	0.5	A

This research addresses a fundamental gap in the domain of a BESS for both modelling and state estimation by introducing a Deep Learning methodology based on Neural Networks (NNs). The key innovation lies in departing from the conventional design starting point of an ECM and traditional Machine Learning algorithms, leveraging the building of several neuron's architecture through KPIs and Transfer learning in battery operating data. The study employs an exhaustive computer science approach within the framework of TensorFlow and Keras as a high-level Application Programming Interface (API), experimentally subjecting the BESS to rigorous testing across diverse conditions, complemented by meticulous performance data acquisition using a programmable DC Electronic Load. This departure from established paradigms not only fills a critical void in battery development but also offers a novel perspective on renewable energy systems and AI methods, particularly in cases where the physical model is already defined.

The paper is structured as follows: Section I provides a systematic review of recent advances in Deep Learning applied to a BESS and the motivation for this research. In Section II, a comprehensive case study on Lithium-ion battery cells is presented in parallel to the experimental process carried out. Section III presents an overview of the implemented NNs, their architecture, parameters, functionalities, and mechanisms. Section IV elucidates the Deep Learning Methodology and its corresponding procedures. The results and performance are given in Section V. Finally, the conclusions drawn from the research and further work are provided in Section VI.

II. CASE STUDY: LEV50N BATTERY CELL TESTING

The BESS corresponds to a battery pack of Mitsubishi i-MiEV, consisting of 88 Lithium-ion battery cells type LEV50N manufactured by GS Yuasa. The battery pack has its origins in the innovative ISEAUTO project, an Estonian autonomous electric vehicle (AEV) from the Tallinn University of Technology (TalTech) campus [16]. However, it is important to note that due to topics of research importance and energy cooperation, the battery pack was dismantled into several battery cell modules. For a comprehensive understanding of the LEV50N cells, Table I provides a summary of the cell parameters.

Importantly, different battery cells were tested from their corresponding modules to experimentally evaluate not only an optimal working status but also a diagnostic. These challenging

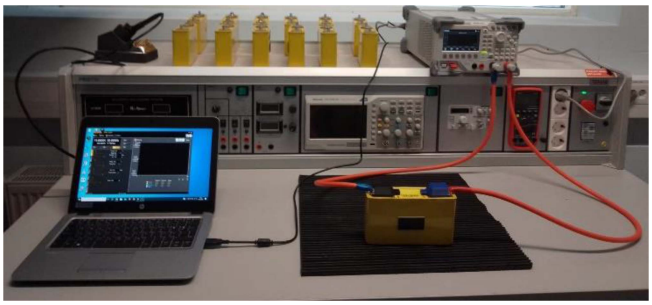

Fig. 1. LEV50 battery test using a programmable DC Electronic load.

tasks constitute central research added value, as the primary objective of this study is to establish a precise Deep Learning methodology for battery modelling and state estimation applied to charging management. The development of this methodology serves as a foundational step for the subsequent experimental validation of NN algorithms from an energy domain perspective.

To accomplish this objective, a series of diverse testing was systematically conducted on the LEV50N battery cells using a programmable DC Electronic Load. These processes were designed to comprehensively explore the battery's performance characteristics and to gain deeper insights into its intricate electrical engineering operation. The programmable DC electronic load has an adjustable current rising speed from 0.001 A/μs to 5 A/μs, readback resolution of 0.1 mV and 0.1 mA, list function that supports editing as many as 512 steps, and dynamic mode up to 30 kHz. In this research, the battery tests consist of a slew rate of 0.001 A/μs, a step duration of 1 second, a frequency of 1 Hz, and a resolution of 0.8%. The battery cell terminals established a connection with a data acquisition system installed on a PC, allowing accurate data collection under various test conditions. Additionally, the battery test functionality of the programmable DC Electronic Load was activated, and operational data was seamlessly integrated into the data acquisition system for a comprehensive evaluation of battery performance. The experimental setup is illustrated in Fig. 1 and the Data acquisition step will be explained in Section IV.

III. Neural Networks Overview

This section presents a brief review of the literature of the different NNs, their corresponding mechanism, architecture, and functionalities, focusing on three categories: 1) Artificial Neural Network (ANN), 2) Convolutional Neural network (CNN) and 3) Recurrent Neural network (RNN), with the aim of familiarizing the reader with the Deep Learning algorithms implemented in the following sections. The reason behind the selection of current NNs is based on the outstanding level of accuracy of Deep Learning algorithms for battery health monitoring, highlighting specific architectures [17], [18], [19], which are computationally and experimentally validated into the understanding of battery state monitoring problems.

It is fundamental to clarify that model parameters are the model's aspects learned from the data during Training. Hyperparameters, on the other hand, are set before Training begins and dictate a model's overall structure and behavior.

There are many hyperparameters that integrate a NN, but among the most important are the number of layers and units, learning rate, activation functions, epochs, and batch size. The number of layers and the size of each layer determine the depth and width of the neural network respectively, affecting the amount of information the model can capture and its suitability. The learning rate regulates the amount of error allocated with which the model weights are updated during training with respect to the loss. Activation functions are nonlinear transformations that transform inputs into outputs and affect the model's ability to capture nonlinear relationships and achieve stability. Finally, an epoch is the total quantity of iterations of all training data in a cycle through the learning algorithm, and the batch size is the number of samples passed to the network at a time [20].

A. Artificial Neural Network

ANNs are computational models whose mechanism is like the functioning of a human nervous system. There are several types of ANNs, which are implemented based on mathematical operations and a set of parameters necessary to determine the predictions. First, the input layer is the initial layer of the network, which takes the independent variables in the form of numbers. Second, the hidden layer, which processes the information received from the input layer. Third, the output layer is responsible for producing results from the calculations applied to data over the network.

In this article, three types of ANN are designed, which are Shallow Neural Network, Deep Neural Network (DNN), and Multilayer Perceptron (MLP). A Shallow Neural Network is defined as an ANN that contains a single hidden layer between the input and output layers, whose limited capacity struggles with complex tasks and different learning patterns. A MLP is usually integrated by three or more interconnected layers, which usually require a short period of training to learn the representations in data and produce an output. In contrast, a DNN has multiple hidden layers stacked between the input and output layers, manifesting a greater ability to learn complex patterns and hierarchical representations of data, allowing them to capture more complex relationships and features [20], [21], [22], [23].

With the objective of testing errors from output nodes to input nodes in an NN, back propagation is a method used in ANNs to calculate the gradient of a loss function with respect to all the weights in the network. The main working framework of an ANN is explained in the following points [20]:

- Random weights are assigned to the NN. The activation rate of the hidden nodes is found using the inputs and the linkages from the input to the hidden layer.
- The activation rate of the output nodes is calculated based on the activation rate of hidden nodes and linkages to the output. The linkages between hidden and output nodes are recalibrated and the error rate of the output node is calculated.
- Using the weights and error found at output node, the error of the hidden nodes is minimized. The weights between the hidden node and input nodes are recalibrated.
- The previous steps are repeated until convergence and the activation rate of the output nodes is obtained using the final linkage weights.

B. Convolutional Neural Network

Depending on the type of connections between the neurons, two categories of architectures can be found, which are feed forward NNs and RNNs. In a feed forward network, information moves in a single direction (forward) from the input nodes, through the hidden nodes, and to the output nodes, excluding cycles or loops in the network [24].

The mechanism of a feed forward NN involves the forward phase and the backward propagation phase. In the forward phase, the input data is fed into the network and propagated through the network. At each hidden layer, the weighted sum of the inputs is calculated and passed through an activation function, which introduces nonlinearity into the model until the output layer is reached and a prediction is achieved. In the backpropagation phase, once a prediction is made, the error is calculated, propagated back through the network, and the weights are adjusted to minimize this error [24].

CNN is a feed forward NN that applies convolutional operations to the input instead of general matrix multiplication, being able to extract features from data with convolutional structures in at least one of its layers. Compared to other type of NNs, a CNN has different layers that conforms its intrinsic architecture, which are briefly summarized as follows [25]:

- *Convolutional Layers:* They apply the convolution operation to the input data, combining the input and filter to create a feature map.
- *Pooling Layers:* They subtract from the feature maps to make the network invariant to small translations, reducing the number of parameters and controlling overfitting.
- *Activation Layer:* Its main function is to apply a nonlinear activation function to the output of the pooling layer, introducing nonlinearity into the model, allowing it to learn more complex representations of the input data.
- *Normalization Layer:* The main objective is to ensure that the activations of each layer are well adapted and avoid overfitting by performing normalization operations, such as batch normalization or layer normalization.
- *Dropout layer:* It ensures that the model does not memorize the training data, but rather generalizes it to new, unseen data, preventing overfitting by randomly removing neurons during training.
- *Dense layer:* After feature extraction from convolutional and pooling layers, it can be used to combine features and make a final prediction; furthermore, the dense layer is usually the final layer and is used to produce the output predictions.

In the Deep Learning methodology, three types of CNNs are designed, which are CNN-1D, CNN-2D and CNN Long Short-Term Memory (CNN-LSTM), the latter being the most used to solve predictions and forecasting problems, having excellent performance in terms of stability and precision compared to standard Machine Learning algorithms [26].

C. Recurrent Neural Network

RNNs have an architecture based on recurrent connections, capable of modeling sequential data for sequence recognition and prediction. RNNs are formed by high-dimensional hidden states with nonlinear dynamics, in which the structure of the hidden states serves as the network's memory and the state of the hidden layer at any given time is conditional on its previous state [27].

The vanishing gradient problem occurs when a zero gradient is obtained and the weights cannot be optimized during the backpropagation process, so the amount of data stored in memory is limited. Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) are types of RNNs designed to handle sequential data. They address the vanishing gradient problem in traditional RNNs by introducing gating mechanisms that allow them to capture long-term dependencies more effectively [28].

LSTM is composed of three gates (forget, input, and output) that control the flow of information to hidden neurons over long periods of time, use more constant errors, and allow RNNs to learn over many more time steps. The corresponding gates of LSTM preserve extracted features from previous time steps regulating not only the amount of data entering at each corresponding time step, but also the number of weights that are optimized [6], [28].

GRU has control units that modulate the flow of information within the unit, but without having separate memory cells. Unlike LSTM, GRU exposes the entire state at each step and computes a linear sum between the existing state and the newly computed state. Hidden layers containing memory cells cover the main functions of GRU networks. Cell state changes and maintenance depend on two gates in the cell: a reset gate and an update gate [6], [27].

A Bidirectional Recurrent Neural Network (BRNN) is a category of RNN that processes input data both forward and backward, which has the property of being trained using all available input information in the past and future of a specific time. The operating mechanism of a BRNN consists of two separate backpropagation phases: one for the forward RNN and one for the backward RNN [29].

During the forward phase of a BRNN, the input sequence is processed by the forward RNN processes like the usual RNN, and the predictions of the output sequence are made; these predictions are subsequently compared to the target output sequence and the error is propagated back through the network to update the weights of the forward RNN. In the backward phase, the RNN processes the input sequence in reverse order and predicts the output sequence. Once both phases are complete, the weights of the forward and backward RNNs are updated based on the errors obtained during the forward and backward passes, respectively. Finally, the process is repeated for multiple iterations until the model converges and the predictions are accurate [29], [30].

The RNNs designed for the Deep Learning methodology are LSTM, GRU, Bidirectional LSTM (BiLSTM) and Bidirectional GRU (BiGRU), whose performance will be compared and analyzed in the following sections.

D. Summary of the Section

Summarizing this section, ANNs are beneficial for solving problems related to tabular data, image data and text data due to their different architecture based on one or multiple network units in each layer, being able to learn non-linear functions, but having difficulties in capture sequential information. On the other hand, RNNs have a recurrent connection on the hidden state whose objective is to ensure that sequential information

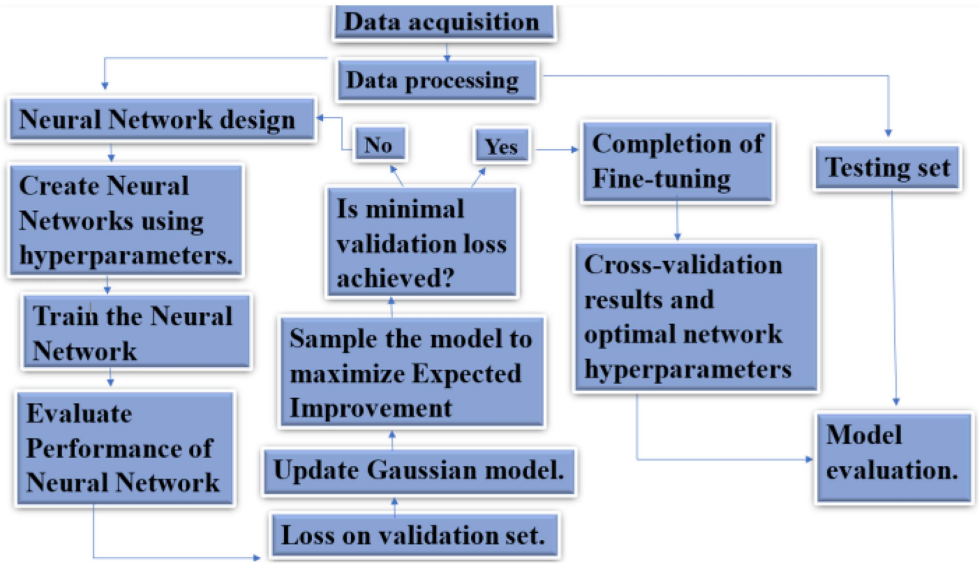

Fig. 2. Flow-chart of the deep learning methodology.

is captured in the input data, being able to solve tasks mainly related to time series, text data and audio data, and having the greatest advantage of reducing computational cost due to the parameter sharing property. Regarding CNNs, the architecture design has more complexity compared to ANNs and RNNs due to filters and kernels to perform convolution operations, resulting in efficient solutions when dealing with image data and sequential inputs, capturing spatial relationships through parameter sharing.

Currently, Deep Learning algorithms show remarkable accuracy in solving different types of problems in the scientific and technological field, however, there is no universal approach that can consider a specific NN more efficient, but depending on the objective, needs and user restrictions, some algorithms are more feasible to adapt in the network architecture. From the BESS perspective, considering battery properties is of vital importance in the experimental behavior, so that the most accurate NN is selected according to the operation and scenarios for battery tests.

The implementation of the different NNs in the following sections will provide a pioneering step to explain and understand the behavior of a BESS during the execution of AI methods, answering the initial research question. It is necessary to highlight that the mechanism and architecture of each NN will determine its viability to adapt the Deep Learning methodology to the various needs of users, experimental tests, battery properties, and operating conditions.

IV. Deep Learning Methodology

The first step in the Deep Learning methodology involves the understanding of the predictors and target variable in the Data acquisition step. This extensive data collection, obtained through meticulous battery tests and measurement techniques, serves as the basis for subsequent analyses, to delve deeper into the BESS's behavior, identify patterns, and ultimately reconstruct its underlying mechanisms. It is necessary to mention that different charging and discharging processes were carried out to collect the training and validation sets, monitoring several C-rates and operating voltage ranges. A more detailed explanation of the final battery tests will be given at the end of this section.

Regarding input features in the Data acquisition step, the Voltage [V], Current [I], Resistance [R], Capacity [mAh], Time [seconds], and Energy [wh] are collected using a programmable DC Electronic Load. Subsequently, in the Data processing step, Exploratory Data Analysis is executed to ensure Feature Engineering and Feature Selection. Initial network architectures are designed to start the Training process, after that, Fine-tuning through Bayesian optimization is implemented. Cross-validation is performed not only to improve algorithm performance but also to prevent bias, underfitting, and overfitting. Finally, optimal network hyperparameters are obtained and tested in new data during Model evaluation. A summary of the Deep Learning methodology is appreciated in Fig. 2.

A. Data Processing and Initialization

Initially, training and validation sets are stored to begin the Data processing step, which mainly consists of Exploratory Data Analysis (EDA), Feature Engineering, and Feature Selection. The purpose of EDA is to find insights that serve in subsequent steps such as cleaning, preparing, and transforming data that will ultimately be used in NN algorithms. Furthermore, EDA is executed by performing initial investigations on data to verify assumptions, discover patterns and detect anomalies with the help of statistics and graphical representations.

Regarding the Feature Engineering process, additional features are obtained based on the input data, which mainly consists of the calculation of Power [P], Open Circuit Voltage (OCV) defined as the target or predicted variable, and SOC. In this specific study, the SOC is obtained using the Coulomb counting, while OCV and Power calculations are performed using an Equivalent Circuit Model approach [10]. A histogram of the Probability Density Function based on OCV, and a correlation

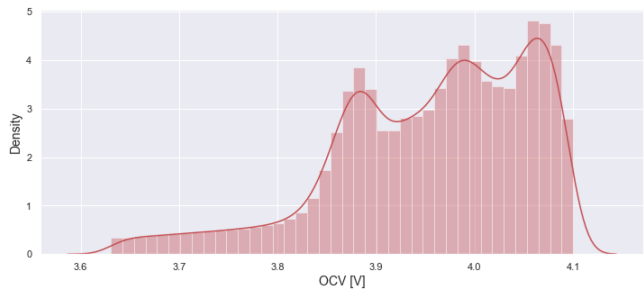

Fig. 3. Histogram of the Probability Density Function based on OCV.

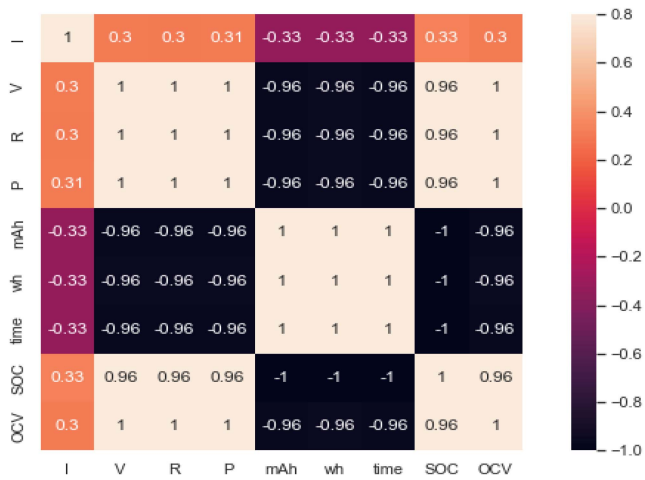

Fig. 4. Correlation matrix after executing Feature Engineering.

matrix of the whole set of features are represented in Figs. 3 and 4 respectively.

It is remarkable to note that there is a strong level of correlation between some of the predictors and the OCV, with Resistance, Power, Voltage and SOC being the most important. Although correlation is a useful statistical tool, identifying the impact of each feature before starting the NN design is a crucial task, so Feature Selection is required to focus on learning patterns in the dataset with fewer samples to make the process more accurate.

The importance of the Feature Selection step relies on improving model performance and identifying KPIs during the algorithm design. As stated previously, EDA results provide information about the emergence of high correlations between several independent variables, leading to multicollinearity, consequently the main goal will be to consider Feature Selection methods that attempt to address multicollinear features.

The Variance Inflation Factor (VIF) is selected as the main criterion to measure the multicollinearity [31], in addition, Ridge Regression and Gradient Boosting [32] are the two main algorithms that will evaluate the performance of the Feature Selection step. A summary of the implemented process is described as follows:

- The VIF is calculated for each feature.
- The feature with the highest VIF score is removed.
- In case some features have the same VIF score, the feature least correlated with the target variable is selected.
- Ridge Regression and Gradient Boosting algorithms are executed, using cross-validation.

TABLE II
FEATURE SELECTION RESULTS

Negative RMSE	Ridge Regression	Gradient Boosting
Best	5.95e-4	3.49e-4
Average	5.96e-4	3.55e-e
Worst	5.97e-4	3.67e-4

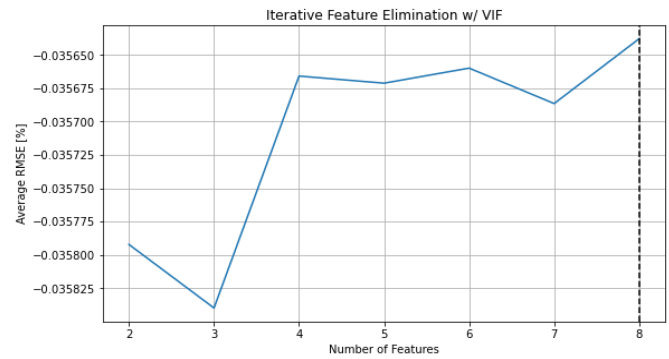

Fig. 5. Gradient Boosting implementation using VIF criterion to achieve feature selection.

- The set of features that results in the best cross-validation score are selected.

Performance of the Feature Selection process is evaluated using the Negative Root Mean Squared Error (RMSE), which compares the Gradient Boosting and Ridge Regression algorithms. Table II summarizes the obtained results.

The highest performance of the Feature Selection process is achieved through Gradient Boosting; therefore, the selected features will be taken using this algorithm. Fig. 5 shows a graphical representation of the implemented VIF criterion to achieve Feature Selection through Gradient Boosting. The different quantity of features is indicated on the X-axis after performing Feature Engineering, while the Y-axis shows the calculated values of the negative RMSE for every feature, the vertical line represents the number of selected features until the maximum level of performance is achieved.

In case the user has a smaller number of features available in the dataset, the performance of the algorithm will decrease according to the Feature Selection process, so executing Feature Engineering also helps in the initial design of the algorithm.

The optimal performance is obtained when the algorithm selects the total number of features, so although there are some features that do not provide a significant level of correlation between the predicted variable, it is necessary to consider them in the NN Design and subsequent steps. Finally, Gradient Boosted Feature Selection [33] is implemented, which ranks all features by importance and selects the top 3, in this specific case capacity, SOC, and voltage, all of them defined as the KPIS and illustrated in Fig. 6. These KPIs represent a high level of statistical correlation in the set of predictors and show intrinsic engineering behavior with the target variable.

Reflecting on the study's approach to Feature Selection, the selected features influence the NNs by increasing the performance of the algorithms based on the number of predictors. As seen in Fig. 5, the highest performance is achieved using the total

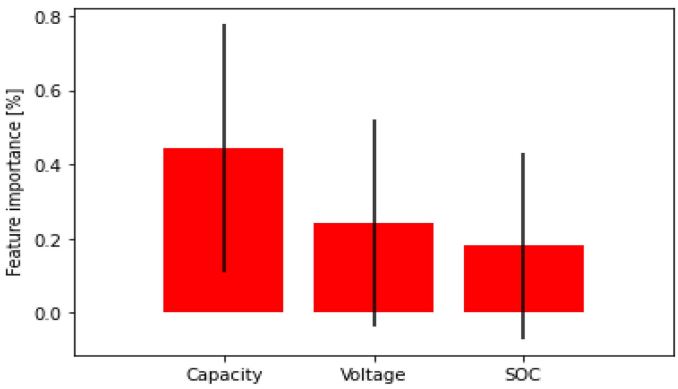

Fig. 6. Implementation of the gradient boosted feature selection to obtain KPIs.

number of independent variables after performing Feature Engineering; however, both interpretability and multicollinearity are added values within the scope of this research to understand the rationale behind the initial network architecture through VIF, Ridge regression and Gradient Boosting. Feature Selection influences the NNs learning to reduce overfitting, so that by selecting the most relevant features, the algorithm is less likely to overfit and can better generalize to new data and achieve optimal prediction capabilities.

The interpretability of the algorithm is initially given by the VIF criterion and subsequently by the KPIs calculated using Gradient Boosted Feature Selection, which not only detects the most relevant features and their relationships with the target variable, but also reflects and allows the user to understand the underlying factors that drive the predictions of the BESS operation.

B. Neural Network Design and Training

After Data processing, training and validation sets were consistently stored and transformed in tensors to ensure alignment with the samples, timesteps and features for execution of next steps using Keras.

The initial network architectures are designed to begin with the implementation of Deep Learning algorithms, so the creation of hyperparameters is the first process to be carried out, however, it is crucial to understand the mechanism of NNs for the purpose of monitoring arguments in Keras that control Training and Validation steps, such as activation functions, metrics, kernel regularizers, optimizers, etc.

Due to the impact on performance and computational efficiency of the Training process, batch size, number of epochs, network units, and learning rate are the selected hyperparameters that will be optimized during Fine-tuning. Regarding additional arguments of the NN in Keras, the ReLu is selected because of its sparsity and beneficial to reduce likelihood of the gradient vanishing.

Regularization techniques are crucial to prevent overfitting or underfitting during the Validation process, so that Dropout and a kernel regularizer are executed in the network design. Furthermore, Adam optimizer is used to minimize the loss and weights, stabilize the Training, and help NNs converge towards optimal solutions.

Before the start of Training, two important techniques are considered to improve the performance of the NNs in subsequent steps, the first considers early stopping to reduce the risk of overfitting, save time and computational resources, which simplifies the model and preserves the best weights. The second implements a learning schedule to avoid exceeding the minimum learning rate and to tune the model parameters more precisely.

Finally, the Training is initialized and explained in the following points:

- Data processing is applied to the training and validation sets. Random weights and biases are randomly generated.
- For each epoch, Forward Propagation is executed through the network and the loss is calculated.
- Once Forward Propagation is completed, the process of Back-Propagation calculates the gradient of loss with respect to weights and biases for each batch. Weights and biases are updated through Adam optimizer.
- Training loss across all batches is calculated.
- Validation losses with their respective biases and weights are calculated at the end of the epoch.

C. Fine-tuning and Bayesian optimization

With its remarkable potential and intricate mechanisms, Deep Learning often depends on fine details. One of those fundamental details for the success of a model is Hyperparameter optimization. In essence, this process systematically searches for the best set of hyperparameters to increase the performance of a model.

Given the vastness of the hyperparameter space, with a nearly infinite number of combinations, finding the optimal set is a challenging and monumental task in every Deep Learning algorithm.

Fine-tuning hyperparameters using various optimization methods is not only beneficial but essential to ensure that models are accurate, efficient, and adaptable. Among the different types of Hyperparameter optimization techniques we find Grid Search, Random Search, and Bayesian optimization, the latter being the most advanced compared to the others and implemented in this research [33].

Bayesian optimization reduces the time required to obtain an optimal set of parameters. When determining the next set of hyperparameters to evaluate, it works by considering previously seen combinations of hyperparameters. Two important concepts are considered into the implementation of Bayesian optimization, which are Exploitation and Exploration. Exploitation means choosing the point with the highest uncertainty in each iteration, while Exploration refers to selecting a point from a region that currently has the best results.

In the context of Bayesian optimization, the main objective is to minimize the loss function. In addition, a model called Gaussian Process is built, which refers to the natural infinite-dimensional analog of the multidimensional Gaussian. A Gaussian Process is used to model the unknown objective function and provides a posterior distribution over the values of the function given the observed data.

To balance Exploration and Exploitation, an acquisition function is implemented, which determines the next point or set of points to evaluate in the search space. Furthermore, based on the current state of the optimization process, an acquisition function

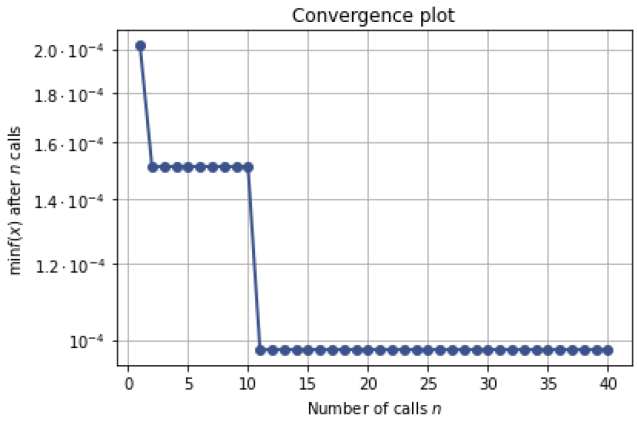

Fig. 7. Convergence plot of a fine-tuning process using bayesian optimization. The X axis refers to the search number, the Y axis indicates the validation loss.

quantifies the possible usefulness or desirability of sampling a particular point, all considering both the predicted values of the surrogate model and the uncertainty associated with those predictions [34].

Expected Improvement is an acquisition function, also defined as a greedy improvement-based heuristic that samples the point offering greatest expected improvement over the current best sampled point. It works by selecting the points that have the potential to improve upon the best-observed value, quantifying over the current best value [35].

The following points provide a summarized explanation of Fine-tuning through Bayesian optimization, using Expected Improvement:

- At first a fitness function is run, which in subsequent steps will provide the performance of the NNs in the dataset based on the selected hyperparameters, so the goal is to find the minimum.
- The Training is executed, and a Gaussian Process is built using an initial set of hyperparameters to know the performance metrics of every NN.
- The Bayesian optimization search starts with the initial values of the hyperparameters. In the initial phase of optimization, the search is explored further and in the later phase, the search focuses on the best regions found.
- The model is sampled to maximize the Expected Improvement and the validation loss is calculated for every search.
- At the end of the process, a set of hyperparameters corresponding to the minimum validation loss is found and the performance metrics are obtained.

The search is completed until a convergence on the minimum validation loss is achieved. An example of a convergence plot using Bayesian optimization is visualized in Fig. 7, showing a minimal validation loss achieved at search number 12 and converged at search number 40.

A plot of all combinations of values for the hyperparameters is also obtained in Fig. 8, which exemplifies a GRU. The vertical axis illustrates the influence of a single dimension on fitness, which is called "Partial Dependence plot" for that specific dimension. It shows how the approximated fitness value changes with different values in that dimension. The yellow regions show areas where the loss on the validation set is lower, unlike the

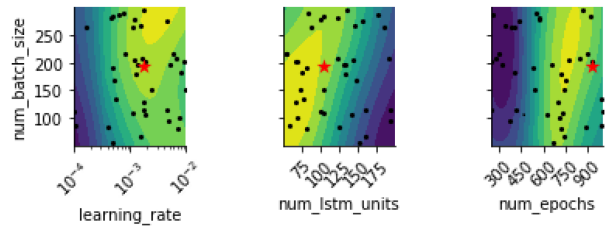

Fig. 8. Partial dependence plot of a fine-tuning process using Bayesian optimization in a LSTM.

darker regions. The star on the graph represents the location where the optimal hyperparameters were found.

It is essential to mention that Fine-tuning through Bayesian optimization was carried out to all type of NNs explained in Section III, so that the ANNs, CNNs, and RNNs highlight different sets of optimized hyperparameters and metrics of performance in the Validation sets, which will be explored in the next subsection.

D. Validation

After the completion of Training and Fine-tuning, the next step consists of selecting a metric to measure the performance of every NN in the Validation set, so that cross-validation is executed. In a Deep Learning algorithm, the goal of Validation is to provide at least approximate performance of the model for data that will appear in the future. Additionally, it is necessary to consider the importance of balancing Underfitting and Overfitting.

Underfitting refers to the model performing poorly on both the training and testing sets. The main cause of Underfitting is that the model does not fit the Training set well or is not trained enough, leading to high bias and low variance. Overfitting indicates the model was excessively fitted during Training, so it performs well on the training set but poorly on new evaluated data resulting in low bias and high variance.

It is worth mentioning that both Underfitting and Overfitting can be identified from the learning curve due to the behavior of training and validation losses in each epoch. An Underfitting plot shows a training loss that remains flat with respect to the evolution of epochs, therefore high levels of loss will be an indicative of the lack of ability to learn the training set. On the other hand, in the case of Overfitting, both training and validation losses will decrease, reach specific values, and start increasing again without any convergence.

Due to its mechanism to expose the model to different subsets and generalize to unknown datasets, K-fold cross-validation is selected, which consists of dividing the data into k subsets (folds) of equal size. In this method, the model is trained on k-1 subsets and tested on the remaining subset, after that, the process is repeated and completed until all subsets are the testing subset at least once. The model performance is the average accuracy of each training/validation round [36].

A learning curve that shows a good fit can be appreciated in Fig. 9, consisting of training losses that decrease to a point of stability, while validation losses decrease throughout the evolution of the epochs and have a small gap with respect to training loss. The numerical difference in gap is appropriate when the range

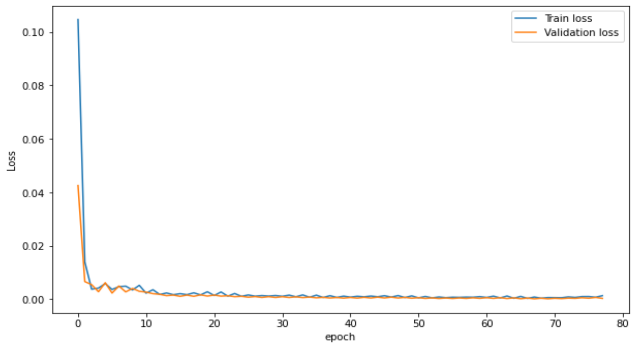

Fig. 9. Learning curve that shows an optimal fit in the validation step. (Blue line) Train loss. (Red line) Validation loss.

TABLE III
ANN PERFORMANCE IN THE VALIDATION STEP

ANN	MSE [%]	MAE [%]
Shallow	0.0030	0.4737
DNN	0.0038	0.4947
MLP	0.0093	0.7273

TABLE IV
CNN PERFORMANCE IN THE VALIDATION STEP

CNN	MSE [%]	MAE [%]
CNN-1D	0.0087	0.8674
CNN-2D	0.0071	0.6523
CNN-LSTM	0.0039	0.4786

TABLE V
RNN PERFORMANCE IN THE VALIDATION STEP

RNN	MSE [%]	MAE [%]
GRU	0.0092	0.7470
LSTM	0.0068	0.6247
BiGRU	0.0050	0.0527
BiLSTM	0.0022	0.0351

is usually less than 1% and nearly zero in an ideal situation, decreasing both training and validation losses; therefore, the Fine-tuning is accomplished, and the optimal performance point of the model has been reached for use on the Testing set.

Finally, the target variable is predicted, and performance metrics are obtained by calculating the Mean Square Error (MSE) and Mean Absolute Error (MAE) for the corresponding NNs. As mentioned in Section III, Shallow, DNN, and MLP are designed in the ANN category. As for CNN, one dimension (CNN-1D), two dimensions (CNN-2D) and CNN-LSTM are implemented. In the case of RNN category, GRU, LSTM, BiLSTM and BiGRU are considered. Tables III, IV, and V show the performance metrics in the Validation step for each ANN, CNN, and RNN respectively.

From the comparison of performance metrics in this problem, it is crucial to evaluate the nature of the datasets. If the dataset contains data points that do not fit the general pattern of the battery tests, it is advisable to select MAEs that provide greater resilience against distortions introduced by outliers. Conversely, if the dataset is relatively clean and without significant outliers, the faster convergence of MSEs could offer an advantage. Regarding the results of the EDA process, there is no significant evidence of outliers in the Data Acquisition step, so no relevant anomalies are found in the dataset, with MSE being more convenient than MAE because it offers faster convergence on the stages. where the error values are relatively small and consistent.

Regarding the numerical values obtained in the performance metrics, the MAE is higher than the MSE, which suggests that the NN have some small errors that accumulate in the MAE, with the variability of operation being the main cause of capturing errors during battery tests. Due to the battery user's needs to predict accurate Charge indicators, MSE is a more optimal performance metric than MAE, not only because it considers large errors instead of small ones, but also because it penalizes models that make significant mistakes.

Considering the continuous advancement in AI methods, the implementation of cross-validation, Regularization, and Fine-tuning through Bayesian optimization is a testimony to the improvement and effectiveness of different categories of NN. Among the most relevant benefits are cost and resource efficiency, which in turn leads to lower costs and less onerous infrastructure requirements in the Model evaluation step.

Contributions of the advanced computer science techniques in this study are summarized in Fig. 10.

The validation results show that BRNNs provide a higher level of performance compared to CNNs and ANNs; however, there are also notable performance metrics in the case of Shallow. A more detailed analysis and explanation will be provided in Section V.

E. Testing

The Testing step consists of an experimental and computational part; in the first, several LEV50N cells were disassembled from different battery modules to test them under various operating conditions, while in the second, the collected datasets were compiled to ensure proper data processing and run Model evaluation.

In the experimental part, sixteen LEV50N cells were tested using a programmable DC electronic load. 43 battery tests were done, in which the operating conditions are: 1) C-rate, 2) operating voltage range, and 3) cut-off voltage range. It is essential to note that the operating conditions were exhaustively measured in various scenarios through the UltraLoad Software for remote operation and monitoring, not only to experimentally evaluate the performance of the cells, but also to test the robustness of the Deep Learning algorithms during the Model evaluation.

As for the computational part, the datasets were collected and the Data Acquisition step was executed, carrying out the corresponding processes explained at the beginning of this section, all using tensor transformation in Tensorflow through Keras. Finally, the Testing set was obtained and prepared to complete the Model evaluation.

The selection of HD5F files as the primary source for NN architectures is based on the format that supports large, complex, and heterogeneous data. Processing and storing optimized

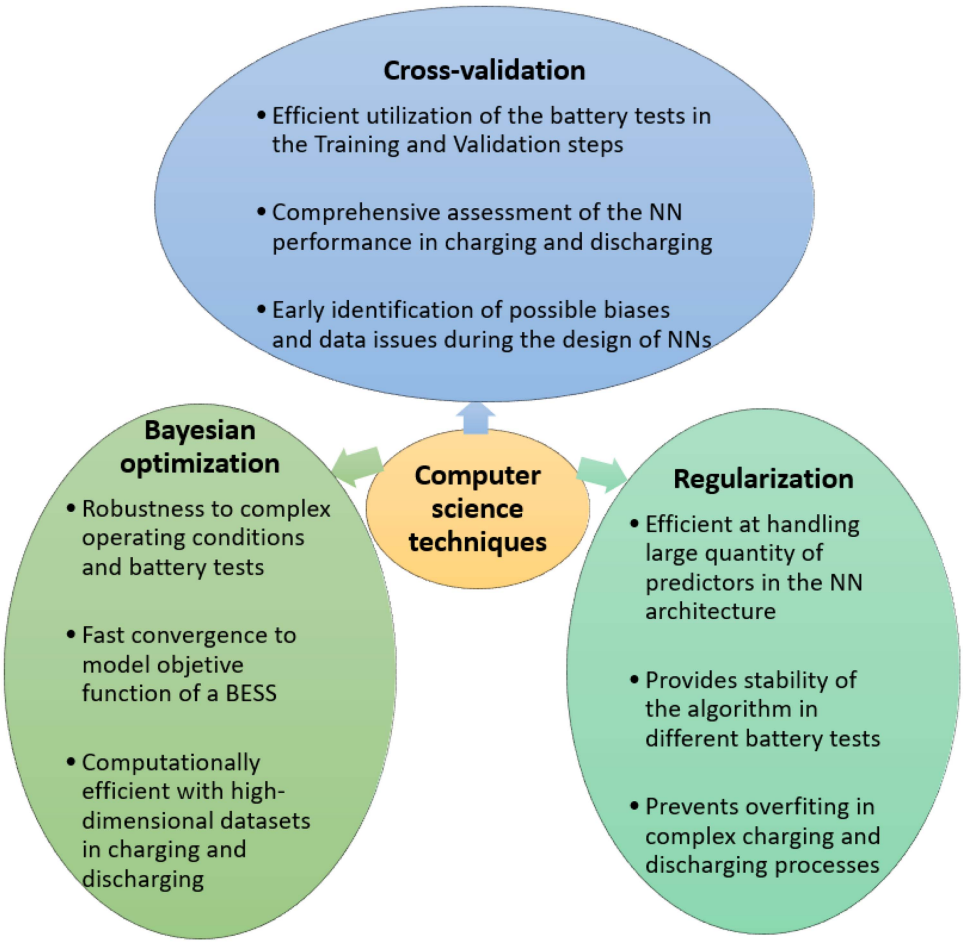

Fig. 10. Main contributions of the computer science techniques implemented in the NN architecture.

NN architectures are key steps in the model evaluation of every developer looking to deliver Machine Learning-based applications. In terms of robustness and effectiveness in battery measurements, H5DF files stored in Keras focuses on debugging speed, code elegance and maintainability, which run faster due to the TensorFlow platform, and are easier to deploy across every platform thanks to the service components.

Considering research results, HDF5-based Big Data integration has been scientifically proven to improve efficient digitization processing in many engineering disciplines, specifically in design and simulation for collaborative applications [37]. Challenges in industry, education and educational teams become increasingly important due to the large amount of data to integrate into software architectures; however, integrating data from HDF5 files into software platforms allows the user to support data sharing and traceability as well as interoperability of tools and workflows [38], [39].

Among the most relevant implications of HD5F files for future AI-driven technology in battery management are: 1) Open format compatible with many programming languages such as C++, C, R, Python, etc., 2) Faster source format to open and read a dataset compared to netCDF4 and Zarr formats [40], 3) Ability to support data splitting and self-describing property that parses through the returned data one element at a time, 4) HDF5 features that achieve parallel I/O performance scalable on a large amount of fragmented and compressed dataset in scientific computing [41].

V. Final Results

In this section, the optimal network hyperparameters after Fine-tuning are provided, to compare and analyze the convergence of each hyperparameter in the NNs.

Regarding the Model evaluation step, a final performance metrics is calculated, and the NNs are evaluated in the different Testing sets to complete the Deep Learning methodology. At the end of this section, a discussion and analysis of the results is carried out, within the framework of the AI methods applied to a BESS.

A. Optimal Network Hyperparameters

The Fine-tuning process was executed through Bayesian optimization, using the Expected Improvement as the acquisition function, after that, K-Fold cross-validation is performed to obtain the optimal hyperparameters of the network.

Table VI shows the optimal set of hyperparameters for each NN, after the successful completion of Fine-tunning and cross-validation.

TABLE VI
OPTIMAL NETWORK HYPERPARAMETERS

Neural Network	Batch size	Learning rate	Units	Epochs
Shallow	298	0.00180	297	653
MLP	217	0.00017	74	913
DNN	172	0.00011	100	889
CNN-1D	216	0.00016	323	992
CNN-2D	202	0.00008	341	760
CNN-LSTM	115	0.00001	156	349
GRU	116	0.00024	153	554
LSTM	100	0.00818	175	922
BiGRU	285	0.00603	91	961
BiLSTM	194	0.00172	105	928

In the context of convergence from the Bayesian optimization, the optimal values of learning rate are similar for the same type of NNs, which complements the mechanism and network architecture in the algorithm design. In terms of units, RNNs show a small numerical difference in optimal values, which differ from CNNs and ANNs. Regarding the number of epochs and the batch size, the convergence point exceeds 500 and 156 in most cases, taking advantage of the benefits of training dynamics mainly due to the early stopping and Regularization techniques.

By analyzing the values of the optimized network hyperparameters, it is notable to identify that batch size values below 202 tend to converge towards sharp minimizers of the training and validation functions, and that sharp minima lead to slower training time. In contrast, batch size values in the range of 216-298 consistently achieve stable convergence and better generalization. The effect of using values of batch size below 64 can cause the network to diverge or converge to a non-optimal minimum.

The numerical values of the learning rate show that higher values represent the network taking larger steps along the error gradient, while a lower learning rate means the network takes smaller steps. Considering the optimized results, when the learning rate is above 0.1 the network may exceed the minimum of the loss function and diverge, resulting in unstable and inaccurate predictions. On the contrary, when the learning rate is below 1e-4, the network may take too long to converge or get stuck in a local minimum, resulting in slow and suboptimal performance.

The speed and quality of the Training step is mainly affected by the optimized number of epochs, so using a low number of epochs below 300 may cause NN to not learn enough from the data and have poor performance. Conversely, using an excessive number of epochs higher than 1000 can cause the NN to memorize the training data and lose its ability to generalize to new, unseen data.

In the case of units, the design of the NN plays the most important role in determining the optimal values, so having many layers and more complex architectures such as BiLSTM, BiGRU, MLP and DNN will obtain a lower number of units compared to Shallow, LSTM and GRU. Considering CNNs, there is a high level of similarity in obtaining optimal unit values, which is based on their intrinsic architecture and mechanism for both CNN-1D and CNN-2D, but which differ from CNN-LSTM.

TABLE VII
ANN FINAL PERFORMANCE IN MODEL EVALUATION

ANN	MSE [%]	MAE [%]	RMSE [%]
Shallow	0.3337	4.1029	4.6023
DNN	0.8236	6.3998	7.1634
MLP	0.3927	4.5391	5.0924

TABLE VIII
CNN FINAL PERFORMANCE IN MODEL EVALUATION

CNN	MSE [%]	MAE [%]	RMSE [%]
CNN-1D	0.1994	3.4304	4.1326
CNN-2D	0.7560	6.5093	7.1899
CNN-LSTM	0.4798	5.1284	5.8794

TABLE IX
RNN FINAL PERFORMANCE IN MODEL EVALUATION

RNN	MSE [%]	MAE [%]	RMSE [%]
GRU	0.5895	5.6717	6.2832
LSTM	0.4529	5.2620	6.0007
BiGRU	0.0041	0.4665	0.5125
BiLSTM	0.0026	0.3721	0.4149

From the BESS perspective, selecting a C-rate is a crucial task that will determine the time it takes the cell to charge or discharge, which will have a direct impact on the Training and Fine-tuning, so the number of batch size and learning rate are the two main hyperparameters intrinsically related to the charging management applications, all due to the number of samples used in a forward and backward propagation through the network. The above statement is because the BESS parameters will change according to different scenarios and user needs, so the network hyperparameters must be robust to evaluate the performance of the model, predict the target variable, and avoid poor generalization based only on a specific dataset.

After completing the Fine-tuning and cross-validation steps, the optimal hyperparameter networks with their respective architectures were processed and saved using the Keras API to create different Hierarchical Data Format (HDF5) files. In the next section, the final performance metrics in the Testing set will be analyzed, compared, and discussed to complete the Deep Learning methodology.

B. Model Evaluation

A total of 43 battery tests on sixteen cells, where each one corresponds to a Python list, were processed, and transformed into a final Testing set using the Keras API in the Tensorflow Framework. After that, the optimized network architectures were evaluated on the Testing set and the MSE, MAE and Root Mean Square Error (RMSE) were calculated to obtain the final performance metrics. Tables VII, VIII and IX show a summary of the results in the Model evaluation step for each type of NN.

Even under different operating conditions and scenarios in which the battery tests were carried out, there is outstanding accuracy with MSE less than 1% and MAE and RMSE less than 7.20% for all types of NN, considering account for the large amount of unseen data evaluated. Considering the performance metrics in the Validation step, Shallow, BiLSTM and

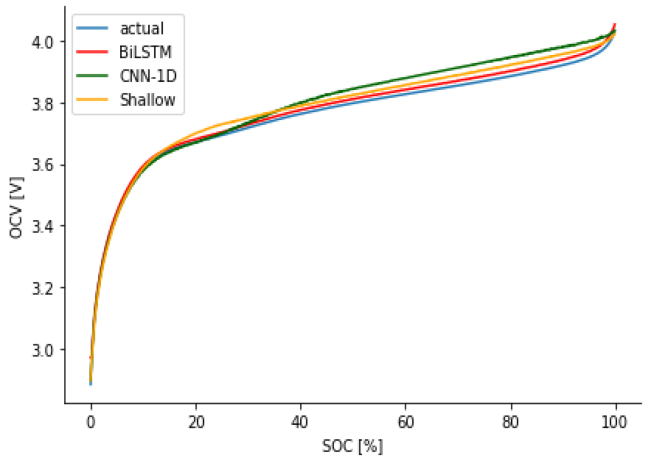

Fig. 11. Graphical representation of the predictions in the model evaluation step, considering ANNs, CNNs, and RNNs.

BiGRU continue to provide the minimum error rate for the ANN and RNN categories. On the contrary, for the CNN category, CNN-LSTM has a slightly lower accuracy than CNN-1D but still higher than CNN-2D.

The interpretation of the results of CNNs is supported by the fact that their main architecture is useful for modeling spatial localities using shared weights in complex tasks, which can lead to complications in capturing long-term dependencies in input sequences such as charging management of a BESS. In the case of ANNs, MLP and DNN show similar performance, however, for this task, the more hidden layers added, the more complex in making predictions, which is explained by the computationally intensive process of Multi-layer feedforward that in consequence can predict the target variable with less precision during fast charging or discharging.

The outstanding performance of RNNs is attributed to their intrinsic property of handling sequential data for sequential modeling and temporal dynamics. However, BRNNs demonstrated that their mechanism is beneficial in tasks where the entire sequence contains past and future time steps, which requires context from both directions to understand the operation of a BESS under different profiles.

Graphical results illustrating the most accurate predictions for each type of ANN, CNN, and RNN are visualized in Fig. 11.

Given the extensive tests conducted on Lithium-ion cells under diverse operating conditions, the NN architecture plays the most important role in determining the battery behavior and the effectiveness of the model. To summarize this section, a comparison of the different NN categories is illustrated in Fig. 12, addressing the contributions of each architecture in improving predictive accuracy and efficiency in battery state estimation, and highlighting their expected impact on the final model's performance. Finally, a brief discussion summarizing the entire Deep Learning methodology will be presented in the next subsection.

C. Discussion

In the initial steps, Feature Engineering and Feature Selection play the most important role, all to provide information about the correlation and data distribution of predictors before algorithm design. It was demonstrated through Gradient Boosted Feature Selection in parallel with VIF, that even a high level of correlation is found, the algorithm shows the maximum level of performance using all features and achieving KPI calculation.

From the BESS domain, including all predictors and Charge indicators allows the user to understand the operating profile and physical interpretation of the battery test, however, not in all cases these indicators are available, which is why the KPI calculation and Feature Selection are required to ensure data processing and gain insight into future model evaluation. Regarding the architecture of the different NNs, a mutual relationship between hyperparameters and predictors was established based on the operating conditions of a BESS, in which the learning rate and the batch size monitor the behavior of the C-rate and the SOC due to different step sizes and samples that update the model weights in the network.

Coding validation and characterization with respect to physical and numerical experiments of the BESS was another point of added value in the research, due to the Bayesian optimization that managed to find the most promising validation scores corresponding to the optimized network architectures, not only considering various battery tests, but also establishing a battery modeling as an objective function.

Regarding the performance metrics in Model evaluation, all NNs provide an accuracy greater than 95%, but the RNNs showed the best accuracy, specifically BiGRU and BiLSTM. These BRNNs are also able to handle variable-length sequences during charge or discharge of a BESS, as they do not depend on a fixed window size or predefined order of inputs. Furthermore, they can learn complex and non-linear patterns during the BESS operation due to the combination of the outputs of two RNNs with different weights and activation functions. This huge mechanism is beneficial for completing even more challenging tasks like forecasting, however, a modification in the network architecture must be executed.

Compared to existing Machine Learning models and physics-based approaches [42], [43], this methodology starts with algorithm design from a beginner level that initially familiarizes the reader with the topic, until reaching an advanced network architecture that can make accurate predictions in the Model evaluation step. Furthermore, considering the OCV analysis, due to the high level of AI methods, NN categories and programming tools of this research, this methodology complements some proposed models that not only focus on the aging mechanisms of a BESS through Data-Driven methods [44], but also in the diagnosis of operational data to construct the OCV-SOC curve [45] and state estimation using DNN [46].

In terms of accuracy, the proposed Deep Learning methodology improves existing methods for OCV prediction and state estimation by not only comparing different NN architectures such as CNN, RNN and ANN, but also explaining the design of these architectures based on the BESS operation. Regarding adaptability, the methodology provides the BESS framework to explain the engineering behavior with the hyperparameters of each NN using Bayesian optimization, this being considered as a pinnacle to initialize a research environment that is capable of unifying virtual and physical entities of a BESS.

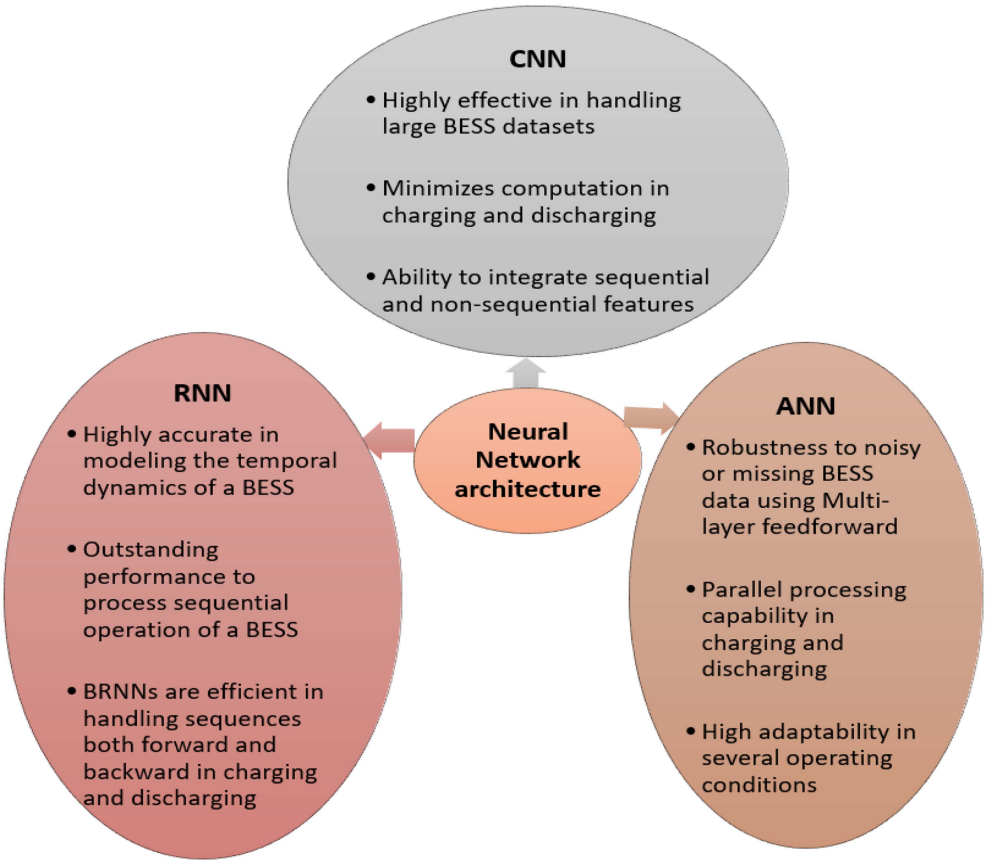

Fig. 12. Main contributions of the different categories of NNs in the model evaluation.

TABLE X
CHALLENGES ADDRESSED BY THE CURRENT METHODOLOGY

Topic	Challenges addressed
Battery technology	• Monitoring the performance of multiple Li-ion cells through extensive battery testing • Validation of experimental procedures for charging management in a R&D environment • Accurate model evaluation based on different NN architectures, scientific computing, and BESS operation
Algorithm design of a BESS	• Virtual entity whose physical representation is based on user needs and KPIs of a BESS • Reducing computational complexity through AI methods • Innovative design using a probabilistic approach, battery properties, and Charge indicators

In the context of the future opportunities of a BESS based on Machine Learning techniques and Data-Driven approaches, the proposed methodology introduces the combination of AI methods within an energy framework to monitor the actual functioning of a BESS based on the user's needs, that vary significantly depending on the available datasets, battery properties, and experimental tests. Table X provides a summary of the challenges associated with predictive maintenance of a BESS addressed by the current methodology.

The Deep Learning methodology represents a crucial step in the future development, verification, and implementation of models in a BESS using AI methods, all to execute optimal predictive maintenance, support business consulting, research and enterprise testing based on renewable energy.

VI. CONCLUSION

In this study, a Deep Learning methodology was employed to validate the experimental testing of Lithium-ion battery cells, marking a crucial step in the state estimation, and charging management applications of a BESS. Extensive battery tests were carried out under diverse conditions and scenarios, accompanied by advanced measurement via UltraLoad Software for remote operation and monitoring. This strategy was realized through the design, development, and implementation of ANNs, CNNs, and RNNs. Feature Selection, Regularization, cross-validation, and Fine-tuning through Bayesian optimization were applied to transform initial networks into several AI models, capable of emulating the intricacies of the battery cells. It was scientifically demonstrated that the design and execution of a BRNN shows the most optimal performance, whose mechanism can explain the behavior of both virtual and physical entities of a BESS, answering the research question. The resulting network architectures, firmly rooted in empirical data, stands as a significant outcome of this research, achieving an outstanding performance that represents a substantial advancement in the development of

AI-powered technology for battery management. The level of adaptability and reliability of this research has been strengthened by a comprehensive methodology that explains the algorithm design from scratch considering Data Science and battery health monitoring perspectives, being a beneficial and understandable testament for different branches of knowledge in the scientific and technological field.

Among the most important novelties of this tremendous methodology are: 1) Reduction of computational complexity in the implementation of Deep Learning algorithms focused on a BESS, 2) Design of different types of NN architectures based on KPIs and hyperparameters in battery cells, 3) Bayesian optimization that provides a framework for combining probabilistic models and Charge indicators for objective battery modeling evaluations, 4) Improving state estimation and KPIs calculation for charging management using AI methods in an energy framework.

This pioneering research represents a pinnacle to unify several branches of knowledge, such as Computer Science, Energy Engineering, Mechatronics, and Software Engineering. The next steps will consist of the deployment through Machine Learning Operations (MLOps), and the incorporation of new architectures based on the multi-head attention mechanism.

Finally, it is of utmost importance to mention that the resources of a research group must facilitate the participation of those resources for other research groups. Our research must facilitate a modest contribution with the aim of promoting ties of cooperation, that is our vocation as researchers, scientists, and scholars.

References

[1] Gilbert Zequera et al., "Overview of battery energy storage systems readiness for digital twins of electric vehicles," *IET Smart Grid*, vol. 6, pp. 5–16, 2023, doi: 10.1049/stg2.1210.

[2] R. A. Gilbert Zequera, A. Rassõlkin, T. Vaimann, and A. Kallaste, "Health and charge indicators for battery energy storage systems in electric vehicles applications," in *Proc. IEEE 20th Int. Power Electron. Motion Control Conf.*, 2022, pp. 427–432, doi: 10.1109/PEMC51159.2022.9962858.

[3] Gilbert Zequera et al., "Modeling battery energy storage systems based on remaining useful lifetime through regression algorithms and binary classifiers," *Appl. Sci.*, vol. 13, 2023, Art. no. 7597, doi: 10.3390/app13137597.

[4] R. A. Gilbert Zequera, A. Rassõlkin, T. Vaimann, and A. Kallaste, "Clustering and outlier analysis for key performance indicators in Battery Energy Storage Systems applications," in *Proc. IEEE 17th Int. Conf. Compat., Power Electron. Power Eng.*, 2023, pp. 1–6, doi: 10.1109/CPE-POWERENG58103.20.

[5] L. Kailong et al., "Lithium-ion battery charging management considering economic costs of electrical energy loss and battery degradation," *Energy Convers. Manage.*, vol. 195, pp. 167–179, May 2019.

[6] R. A. Gilbert Zequera, A. Rassõlkin, T. Vaimann, and A. Kallaste, "Data science-based techniques for modelling and diagnostics of battery cells based on end-of-life criteria," in *Proc. IEEE Int. Conf. Elect. Drives Power Electron.*, 2023, pp. 1–6, doi: 10.1109/EDPE58625.2023.10274007.

[7] A. Mathew, P. Amudha, and S. Sivakumari, "Deep Learning techniques: An overview," in *Proc. Adv. Mach. Learn. Technol. Appl.*, 2021, pp. 599–608, doi: 10.1007/978-981-15-3383-9_54.

[8] X. Du, Y. Cai, S. Wang, and L. Zhang, "Overview of deep learning," in *Proc. IEEE 31st Youth Academic Annu. Conf. Chin. Assoc. Automat.*, 2016, pp. 159–164, doi: 10.1109/YAC.2016.7804882.

[9] G. dos Reis, C. Strange, M. Yadav, and S. Li, "Lithium-ion battery data and where to find it," *Energy AI*, vol. 5, 2021, Art. no. 100081, doi: 10.1016/j.egyai.2021.100081.

[10] J. Tian, C. Chen, W. Shen, F. Sun, and R. Xiong, "Deep learning framework for lithium-ion battery state of charge estimation: Recent advances and future perspectives," *Energy Storage Mater*, vol. 61, 2023, Art. no. 102883.

[11] F. Mohammad, D.-K. Kang, M. A. Ahmed, and Y.-C. Kim, "Energy demand load forecasting for electric vehicle charging stations network based on ConvLSTM and BiConvLSTM architectures," *IEEE Access*, vol. 11, pp. 67350–67369, 2023, doi: 10.1109/ACCESS.2023.3274657.

[12] X. Huang, D. Wu, and B. Boulet, "Ensemble learning for charging load forecasting of electric vehicle charging stations," in *Proc. IEEE Electric Power Energy Conf.*, 2020, pp. 1–5, doi: 10.1109/EPEC48502.2020.9319916.

[13] Y. Li, S. He, Y. Li, L. Ge, S. Lou, and Z. Zeng, "Probabilistic charging power forecast of EVCS: Reinforcement learning assisted deep learning approach," *IEEE Trans. Intell. Veh.*, vol. 8, no. 1, pp. 344–357, Jan. 2023, doi: 10.1109/TIV.2022.3168577.

[14] X. Xiong and L. Zhou, "A method of short-term load forecasting At electric vehicle charging stations through combining multiple deep learning models," in *Proc. IEEE 2nd Asia Power Elect. Technol. Conf.*, 2023, pp. 740–744, doi: 10.1109/APET59977.2023.10489548.

[15] Z. Yang, T. Hu, J. Zhu, W. Shang, Y. Guo, and A. Foley, "Hierarchical high-resolution load forecasting for electric vehicle charging: A deep learning approach," *IEEE J. Emerg. Sel. Topics Ind. Electron.*, vol. 4, no. 1, pp. 118–127, Jan. 2023, doi: 10.1109/JESTIE.2022.3218257.

[16] A. Rassõlkin, R. Sell, and M. Leier, "Development case study of first Estonian self-driving car ISEAUTO," *Sci. J. Riga Tech. Univ.-Elect., Control Commun. Eng.*, vol. 14, no. 1, pp. 81–88, 2018.

[17] J. Tian et al., "Battery state-of-charge estimation amid dynamic usage with physics-informed deep learning," *Energy Storage Mater.*, vol. 50, pp. 718–729, 2022.

[18] J. Tian et al., "State-of-charge estimation of $LiFePO_4$ batteries in electric vehicles: A deep-learning enabled approach," *Appl. Energy*, vol. 291, 2021, Art. no. 116812.

[19] D.-W. Chung, J.-H. Ko, and K.-Y. Yoon, "State-of-charge estimation of lithium-ion batteries using LSTM deep learning method," *J. Elect. Eng. Technol.*, vol. 17, no. 3, pp. 1931–1945, 2022.

[20] M. Islam, G. Chen, and S. Jin, "An overview of neural network," *Amer. J. Neural Netw. Appl.*, vol. 5, no. 1, pp. 7–11, 2019, doi: 10.11648/j.ajnna.20190501.12.

[21] S. Ansari, A. Ayob, M. S. Hossain Lipu, A. Hussain, and M. H. M. Saad, "Multi-channel profile based artificial neural network approach for remaining useful life prediction of electric vehicle lithium-ion batteries," *Energies*, vol. 14, 2021, Art. no. 7521, doi: 10.3390/en14227521.

[22] J. Qiao, X. Liu, and Z. Chen, "Prediction of the remaining useful life of lithium-ion batteries based on empirical mode decomposition and deep neural networks," *IEEE Access*, vol. 8, pp. 42760–42767, 2020, doi: 10.1109/ACCESS.2020.2977429.

[23] J. Qiao, X. Liu, and Z. Chen, "Prediction of the remaining useful life of lithium-ion batteries based on empirical mode decomposition and deep neural networks," *IEEE Access*, vol. 8, pp. 42760–42767, 2020, doi: 10.1109/ACCESS.2020.2977429.

[24] M. H. Sazli, "A brief review of feed-forward neural networks," *Commun. Fac. Sci. Univ. Ankara Ser. A2-A3: Phys. Sci. Eng.*, vol. 50, no. 1, pp. 11–17, 2006, doi: 10.1501/commua1-2_0000000026.

[25] Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 12, pp. 6999–7019, Dec. 2022, doi: 10.1109/TNNLS.2021.3084827.

[26] J. Song et al., "Predicting hourly heating load in a district heating system based on a hybrid CNN-LSTM model," *Energy Buildings*, vol. 243, 2021, Art. no. 110998.

[27] H. Salehinejad et al., "Recent advances in recurrent neural networks," 2017, *arXiv:1801.01078*.

[28] R. M. Schmidt, "Recurrent neural networks (rnns): A gentle introduction and overview," 2019, *arXiv:1912.05911*.

[29] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[30] O. Adigun and B. Kosko, "Bidirectional backpropagation," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 50, no. 5, pp. 1982–1994, May 2020, doi: 10.1109/TSMC.2019.2916096.

[31] R. Tay, "Correlation, variance inflation and multicollinearity in regression model," *J. Eastern Asia Soc. Transp. Stud.*, vol. 12, pp. 2006–2015, 2017.

[32] Z. Xu et al., "Gradient boosted feature selection," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 522–531.

[33] R. Turner et al., "Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020," in *Proc. NeurIPS Competition Demonstration Track*, 2021, pp. 3–26.

[34] J. Wu et al., "Hyperparameter optimization for machine learning models based on Bayesian optimization," *J. Electron. Sci. Technol.*, vol. 17, no. 1, pp. 26–40, 2019.
[35] C. Qin, D. Klabjan, and D. Russo, "Improving the expected improvement algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5387–5397.
[36] I. K. Nti, O. Nyarko-Boateng, and J. Aning, "Performance of machine learning algorithms with different K values in K-fold cross-validation," *J. Inf. Technol. Comput. Sci.*, vol. 6, pp. 61–71, 2021.
[37] S. Ambatipudi and S. Byna, "A comparison of hdf5, zarr, and netcdf4 in performing common i/o operations," 2022, *arXiv:2207.09503*.
[38] M. Folk et al., "An overview of the HDF5 technology suite and its applications," in *Proc. EDBT/ICDT 2011 Workshop Array Databases*, 2011, pp. 36–47.
[39] M. Zhang et al., "A database for developing machine learning based disruption predictors," *Fusion Eng. Des.*, vol. 160, 2020, Art. no. 111981.
[40] M. A. Broecker, M. Siggel, and S. Reitenbach, "Integration of large data based on HDF5 in a collaborative and multidisciplinary design environment, Part A: Methodology and implementation," in *Proc. AIAA SCITECH 2024 Forum*, 2024, p. 0482.
[41] S. Lee et al., "A case study on parallel HDF5 dataset concatenation for high energy physics data analysis," *Parallel Comput.*, vol. 110, 2022, Art. no. 102877.
[42] Y. Zhang et al., "In-situ battery life prognostics amid mixed operation conditions using physics-driven machine learning," *J. Power Sources*, vol. 577, 2023, Art. no. 233246.
[43] M.-K. Tran et al., "Python-based scikit-learn machine learning models for thermal and electrical performance prediction of high-capacity lithium-ion battery," *Int. J. Energy Res.*, vol. 46, no. 2, pp. 786–794, 2022.
[44] B. Pan et al., "Aging mechanism diagnosis of lithium ion battery by open circuit voltage analysis," *Electrochimica Acta*, vol. 362, 2020, Art. no. 137101.
[45] C. Chen et al., "A novel data-driven method for mining battery open-circuit voltage characterization," *Green Energy Intell. Transp.*, vol. 1, no. 1, 2022, Art. no. 100001.
[46] S. Su, W. Li, J. Mou, A. Garg, L. Gao, and J. Liu, "A hybrid battery equivalent circuit model, deep learning, and transfer learning for battery state monitoring," *IEEE Trans. Transp. Electrific.*, vol. 9, no. 1, pp. 1113–1127, Mar. 2023, doi: 10.1109/TTE.2022.3204843.

Rolando Antonio Gilbert Zequera (Student Member, IEEE) was born in Veracruz, Mexico, in 1995. He received the B.Sc. degree in engineering physics from Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Monterrey, Monterrey, Mexico, in 2018, and the M.Sc. degree in engineering and applied sciences, achieving honors mention from Mines Paris-PSL, Paris, France, in 2021. He is currently working toward the Ph.D. degree with the Department of Electrical Power Engineering and Mechatronics, Tallinn University of Technology, Tallinn, Estonia. He has a prestigious education and professional experience in several countries, such as Mexico, USA, France, Norway, Germany, and Estonia. His research interests include deep learning, machine learning, digital twins, and energy industry.

Viktor Rjabtšikov (Member, IEEE) received the bachelor's and master's degrees in electrical engineering in 2018 and 2020, respectively. He is currently working toward the Ph.D. degree with the Department of Electrical Power Engineering and Mechatronics, Tallinn University of Technology, Tallinn, Estonia. His primary research interests include digital twin technology within the automotive industry, the study of physical systems, system modeling, and propulsion systems.

Anton Rassõlkin (Senior Member, IEEE) born in Tallinn, Estonia, in 1985. He received the B.Sc., M.Sc., and Ph.D. degrees from the Tallinn University of Technology, Tallinn, Estonia, in 2008, 2010, and 2014, respectively, and the Dipl.-Ing. degree in automation from the University of Applied Science Giessen-Friedberg, Giessen, Germany. He holds a strong academic background in electric drives and power electronics. He currently holds a tenure track Professor position in mechatronics with the Department of Electrical Power Engineering and Mechatronics, Tallinn University of Technology (TalTech). His primary research interests include mechatronics and electrical drives, with a particular focus on applications in electric transportation and autonomous vehicles, the development of their digital twins. He is recognized as an active member of the Estonian Society of Moritz Hermann Jacobi.

Toomas Vaimann (Senior Member, IEEE) was born in Pärnu, Estonia, in 1984. He received the B.Sc., M.Sc., and Ph.D. degrees in electrical engineering from the Tallinn University of Technology, Tallinn, Estonia, in 2007, 2009, and 2014, respectively. He is currently a Researcher Professor with the Department of Electrical Power Engineering and Mechatronics, Tallinn University of Technology. He is the President of Joung Estonian Academy of Science. He is working in several companies as an Electrical Engineer. His main research focuses on the diagnostics of electrical machines. He is a member of IEEE (S'11-M'14-SM'20), Estonian Society of Moritz Hermann Jacobi, and Estonian Society for Electrical Power Engineering.

Ants Kallaste (Senior Member, IEEE) was born in Pärnu, Estonia, in 1980. He received the B.Sc., M.Sc., and Ph.D. degrees in electrical engineering from the Tallinn University of Technology, Tallinn, Estonia, in 2004, 2006, and 2013, respectively. He is currently a Professor of electrical machines with the Department of Electrical Power Engineering and Mechatronics, Tallinn University of Technology. In addition, he is holding the position of Head of Chair of Electrical Machines Research Group. He has spent several years in private engineering companies and visited numerous other research institutions. Amongst other research activities, he is involved with expertise and consultations of private companies in the field of electrical machines, drives, and their diagnostics. He is a Senior Researcher of IEEE and member of Estonian Society of Moritz Hermann Jacobi.